Overview

Phoenix Security's REST API is the primary form of integration between customer's systems and the platform. The API allows organizations to interact with their data and processes inside of Phoenix Security.

This document contains a detailed definition of the API flows and endpoints. In particular, it covers:

- Authentication: How do you authenticate clients against the API?
- Managing Assets: list, create, and update any type of asset in the platform.
- Managing vulnerabilities: Create and update lists of vulnerabilities and their details.

Over time the API endpoint coverage will expand to cover areas like Users, Teams, etc.

Table of Contents

Overview 1
Authentication 3
Client ID and Secret 3
Base URL 4
Obtaining Access Tokens 5
Using Access Tokens 5
Asset Management Endpoints 7
Asset List 7
Get Asset 9
Add Tags to Assets 10
Vulnerability Management Endpoints 11
Vulnerability List 11
Get Vulnerability 13
Import Endpoints 15
Import Assets 15
Applications and Environments Endpoints 18
Applications/Environments List 18
Single Application/Environment 19
Get Application/Environment Posture 20
Create Application/Environment 21
Update Application/Environment 22
Add Tags to Application/Environment 23
Remove Tags from Application/Environment 24
Add responsible Users to an Application/Environment 25
Add Deployment Link to an Application 26
Link Repository to an App/Env 27
Identifying Applications/Environments by Name 28
Components and Services Endpoints 29

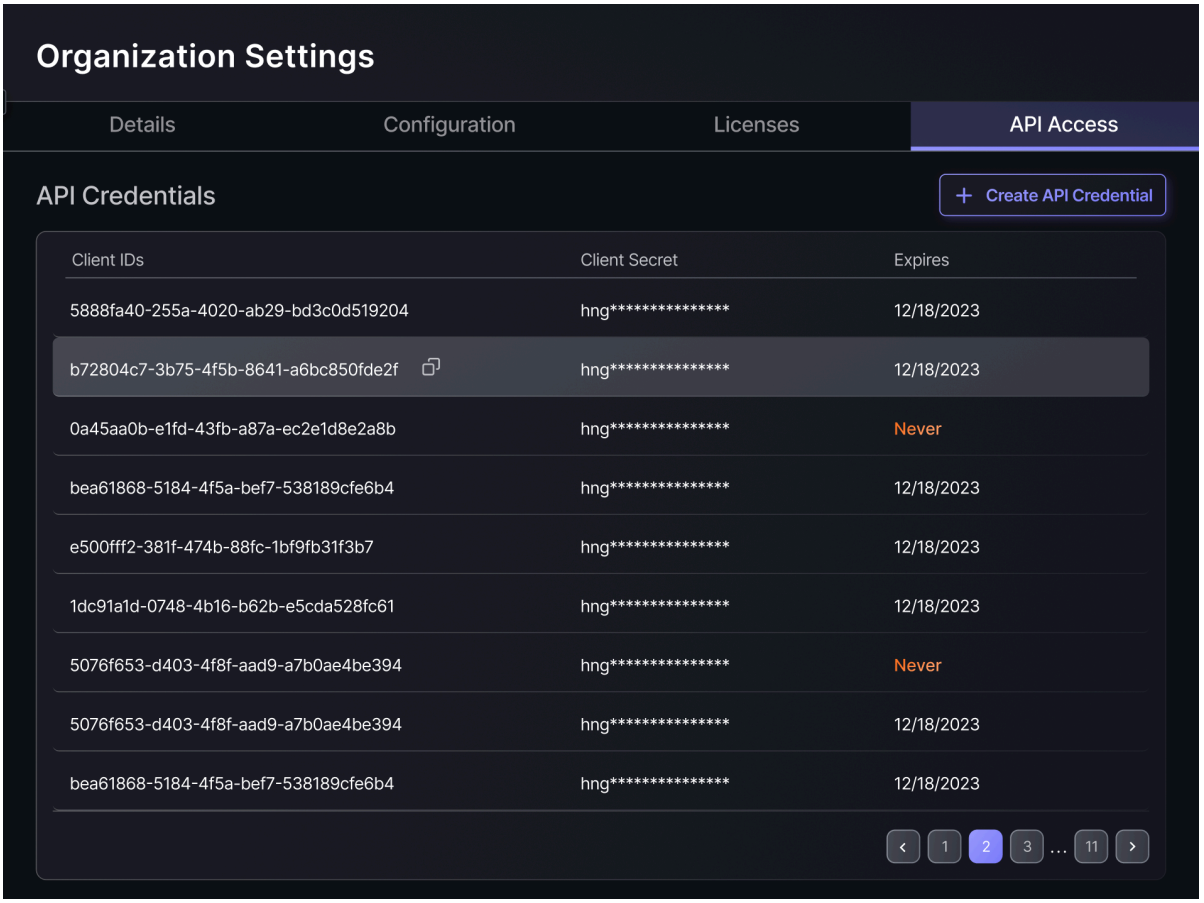
Component/Services List 29
Single Component/Service 29
Get Component Posture 30
Create Components/Services 32
Update a Component/Service 33
Remove Tags from Component/Service 34
Asset Association Rules API Endpoints 35
Add Asset Association Rules 35
Team API Endpoints 37
List Teams 37
Single Team 37
Get Team Members 38
Remove Team Member 38
Create Team 38
Add users to a Team 39
Auto-link Apps/Envs to the Team by tags 40
Remove applications auto-link tags 41
Remove auto-link applications to the team by tags 41
Enable auto-linking Apps/Envs to a Team by team Members 42
Disable auto-linking Apps/Envs to a Team by team Members 42
Auto-link Components to Teams by Tags 43
Disable auto-linking Components to Teams by Tags 44
Remove team's auto-link Tags 44
Identifying Teams by Name 45
Users Endpoints 46
List Users 46
Create User 47

Authentication

Access to the Phoenix Security REST API is controlled by API Client ID and Secret. However, requests to the API endpoints are not individually authenticated by those credentials. Instead, clients must use their ID and Secret to obtain time-limited access tokens - then they can use the access token to authenticate each API request.

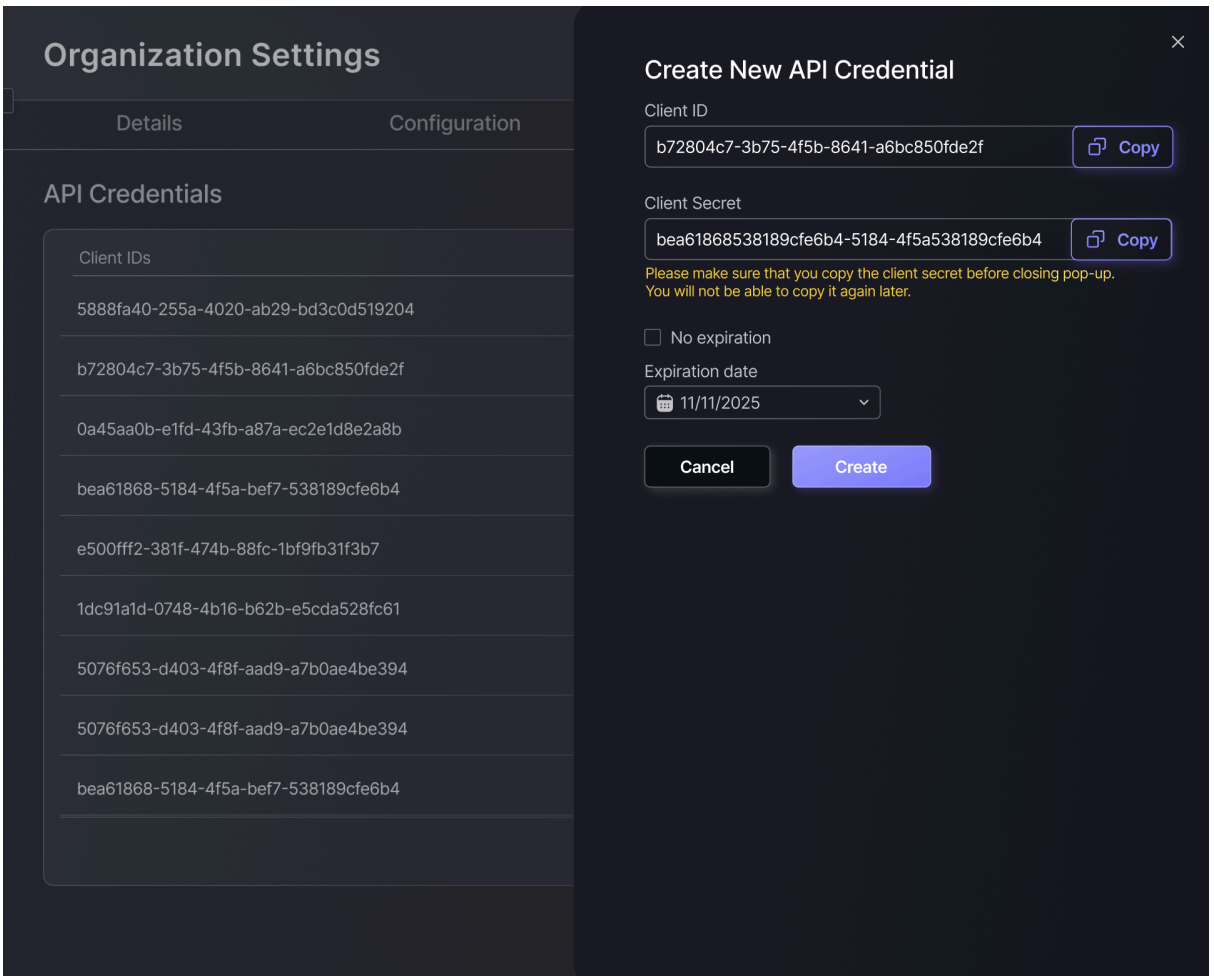
Client ID and Secret

Customers can generate one or more sets of API credentials in the Phoenix Security platform (Organisation > API Access). Each of those sets can be managed independently so that potentially compromised credentials can be deactivated or removed from the platform.



API Credential Management Phoenix Security

When users create API credentials in the UI they will only be able to see and copy the API Secret at the time of creation. This secret is stored in an encrypted form and can't be decrypted again. It's important that users copy and safely store the secret at the time of creation.



API Credential Generation Phoenix Security

The Client ID is a "username" that allows for identification of the secret that needs to be checked - it can be stored in clear text for easy lookup. The Client Secret should be stored in an encrypted (hashed) format so that it's not possible to decrypt it but it's possible to check if a clear text string matches the encrypted secret - like it works with password storage and check.

Base URL

The endpoint details in this document only include the API endpoint paths, without the base domain. The Phoenix Security base URL for SaaS production is **https://api.securityphoenix.cloud**.

For non production/demo https://api.demo.appsecphx.io/ and https://api.poc1.appsecphx.io for enterprise PoC.

If you have a dedicated Enterprise deployment with its own URL in the form https://**mycompany**.securityphoenix.cloud your API domain would be https://**api.mycompany**.securityphoenix.cloud

If you are using any other environment, please check with Support for the correct base URL.

Obtaining Access Tokens

Before clients can make requests to the API they need to obtain an Access Token. In order to do so, clients need to send a request to the tokens endpoint and include an HTTP Basic Auth header with the Client ID (username) and Secret (password).

GET /v1/auth/access_token

Headers:

```
Authorization: Basic <client ID and secret>
```

Responses

Incorrect credentials: **401 Unauthorized**

Correct credentials: **200 OK**

```
{
  "token": string,
  "expiry": number // Time of token expiration as Unix timestamp

}
```

- **token**: An opaque (Bearer) token to be included in subsequent requests to the API.
- **expiry**: The time of expiration of the token in Unix timestamp format. Due to server time shift, clients should not rely on this timestamp alone, and should handle "token invalid or expired" responses as appropriate.

Using Access Tokens

All requests to the Phoenix Security API - except for the */v1/access_token* endpoint - should **include an access token** as part of the request. This is done through an Authorization/Bearer HTTP header.

```
Authorization: Bearer <access token>
```

If the token is missing, it's incorrect or it has expired, the request would return an **401 Unauthorized** code. The response body might, or might not, contain additional details.

API Response Codes

Requests to the API will receive a response with one of the following status codes.

Status Code	Verbs	Meaning	Suggested Action
200 OK	GET	The request succeeded and the response body contains the result of the request in JSON format.	You can use the results of the request.
201 Created	POST, PUT?	The new item was created successfully.	

204 No Content	DELETE	The item was successfully deleted; the response doesn't contain any further information.	
401 Unauthorized	All	The access token sent with the request is wrong or has expired.	Obtain a new access token and retry the request.
404 Not Found	GET	The items accessed by the request doesn't exist - or the API credentials don't have access to it.	If the item ID was returned by another API endpoint, contact support with details.
500 Internal Error	All	There was a problem executing your request. This is a rare response and should not happen normally; however, clients should be ready to handle it.	The problem might be transient or permanent. Clients are advised **not** to retry indefinitely.
503 Service Unavailable	All	This error means that the API service is not reachable at this point in time, but it's likely to become available shortly.	Clients are advised to retry, for a limited number of times, if that suits their flow.

Asset Management Endpoints

This group of endpoints allow you to access the assets registry in order to query, insert and update assets.

Asset List

In order to obtain the asset list you can use the following request:

POST /v1/assets

Query Parameters

Request page parameters:

```
?pageNumber=0&pageSize=20
```

Requests parameters are sent in the POST body.

```
{
	"requests": [
		"type": "string",
		"applicationEnvironmentId": "string (UUID)",
		"componentServiceId": "string (UUID)",
		"onlyUnassigned": true | false
		"filters": [{
			"key": "string",
			"locality": ["string""],
			"tagKeys": ["string""],
			"tagValues": ["string],
			// CLOUD ===============
			"resourceType": ["string""],
			"providerAccountId": ["string"],
			"vpc": ["string"],
			"subnet": ["string"],
			"region": ["string"]
			// INFRA =========
			"cidr": "string",
			"hostnames": ["string"],
			"osNames": ["string"],
			"netbios": ["string"],
			"macAddress": ["string"]
			//WEB/API ============
			"cidr": "string",
			"fqdn": ["string"]
			// SAST/SCA/Container ===
			"repository": ["string"],
			"build": ["string"],
			"dockerfile": ["string"],
			"scannerSource": ["string"]
		}]
	]
}
```

where:

- **type** the type of assets to return; empty or missing means *any type.* Required.
- **applicationEnvironmentId** is the Phoenix ID for the Application or Environment to return assets from. Optional.
- **componentServiceId** is the Phoenix ID for the Component (App) or Service (Env) to return assets from; if this value is present "applicationEnvironmentId" will be ignored. Optional.
- **onlyUnassigned** can be used to fetch only assets that are not assigned to any Environment (*true*), i.e. they are assigned to a Default Environment. If missing it defaults to *false* (include all). Optional.

Response

A successful request would return a JSON document with the following format:

```
{
  "content": [
    {
      "id": "string (UUID)",
      "key": "string",
      "type": "string",
      "resourceType": "string",
      "locality": "string",
      "data": [
        {
          "source": "string",
          "attributes": {
            "<key>": "value",
          }
        }
      ],
      "tags": [
         "key": "string",
         "value": "string"
      ]
    }
  ],
  "first": "boolean",
  "last": "boolean",
  "pageNumber": "number",
  "pageNumberOfElements": "number",
  "pageSize": "number",
  "totalElements": "number",
  "totalPages": "number"
}
```

Get Asset

In order to obtain the details of an asset you can use the following request:

GET /v1/assets/<asset-id>

Path Parameters

- **asset-id**: This is the internal ID of the asset, as returned in the "id" field of the asset list request's response.

Response

A successful request would return a JSON document with the following format:

```
{
      "id": "string (UUID)",
      "key": "string",
      "type": "string",
      "resource_type": "string",
      "locality": "string"
      "data": [
          "source": "string",
          "attributes": {
             <attr_name> : <attr_value>
          }
      ]
      "tags": [
          "key": "string",
          "value": "string"
      ]

}
```

Here `<attr_name>` and `<attr_value>` are placeholders for the different number and type of attributes that each **type of asset** contains.

Add Tags to Assets

Tags can be added to assets using the following requests to either add them to a single asset or to multiple assets, respectively.

PUT /v1/assets/<asset-id>/tags

PUT /v1/assets/tags

Path Parameters

- **asset-id**: This is the internal ID of the asset, as returned in the "id" field of the asset list request's response.

Parameters

Requests parameters are sent in the PUT body:

```
{
   "tags":[
      {
         "key":"string",
         "value":"string"
      }
   ],
   "assetIds":["uuid"]
}
```

where:

- **tags[]**: Array of objects. A `key` is optional, `value` is required
- **assetIds[]**: Array of UUID strings. Required in the version of the endpoint without the *<asset-id>* in the path; otherwise ignored. The list of asset IDs that the tags will be added to.

Vulnerability Management Endpoints

This group of endpoints allow you to access the vulnerability registry in order to query, insert and update vulnerabilities.

Vulnerability List

In order to obtain the vulnerability list you can use the following request:

POST /v1/vulnerabilities

Query Parameters

Request page parameters:

```
?pageNumber=0&pageSize=20
```

Requests parameters are sent in the POST body.

```
{
        "applicationEnvironmentId": "string (UUID)",
        "componentServiceId": "string (UUID)",
        "assetId": "string (UUID)",
        "assetAssignedStatus": "string",
        "assetLocalities": ["string"],
        "type": "string",
        "scannerTypes": ["string"],
        "status": ["string"],
        "suggestedToFix": "boolean",
        "zeroDayAlert": "boolean",
        "dateFrom": "date",
        "dateTo": "date"
        "severityScoreFrom": "string",
        "severityScoreTo": "string",
        "cves": ["string"],
        "epssScoreFrom": "string",
        "epssScoreTo": "string",
        "workflowTicketStatuses": ["string"],
        "slaBreach": "string",
        "workflowTicketSlaBreach": "string",
        "tagKeys": ["string"],
        "tagValues": ["string"],
        "tagCategories": ["string"]
}
```

where:

- **applicationEnvironmentId** is the Phoenix ID for the Application or Environment to return vulnerabilities from
- **componentServiceId** is the Phoenix ID for the Component (App) or Service (Env) to return vulnerabilities from;
- **assetId** is the Phoenix ID for the Asset to return vulnerabilities from
- **assetAssignedStatus** Allowed values: `ASSIGNED, UNASSIGNED`
- **assetLocalities** Allowed values: array of `[INTERNAL, EXTERNAL, DMZ]`
- **type** Allowed values: `WEB, CLOUD, FOSS, SAST, CONTAINER, INFRA`
- **scannerTypes** Allowed values: array of scanner codes (request list)
- **status** Allowed values: array of `[OPEN, FALSE_POSITIVE]`
- **suggestedToFix** Allowed values: `true, false`
- **zeroDayAlert** Allowed values: `true, false`
- **dateFrom** Allowed format: `yyyy-MM-dd`
- **dateTo** Allowed format: `yyyy-MM-dd`
- **severityScoreFrom** Allowed values: `0 - 1000`
- **severityScoreTo** Allowed values: `0 - 1000`
- **cves** Allowed values: array of: `[must start from CVE- prefix]`
- **epssScoreFrom** Allowed values: `0.0 - 1.0`
- **epssScoreTo** Allowed values: `0.0 - 1.0`
- **workflowTicketStatuses** Allowed values: array of `[string]`
- **tagKeys** Allowed values: array of `[string]`
- **tagValues** Allowed values: array of `[string]`
- **tagCategories** Used to narrow down the search by **tagKeys/tagValues**. Allowed values: array of `[TECH_STACK, ALERT, COMPLIANCE, BUSINESS_UNIT, GENERAL]`

Response

A successful request would return a JSON document with the following format:

```
{
   "content":[
      {
         //see fields in the vulnerability item
      }
   ],
  "first": "boolean",
  "last": "boolean",
  "pageNumber": "number",
  "pageNumberOfElements": "number",
  "pageSize": "number",
  "totalElements": "number",
  "totalPages": "number"

}
```

Get Vulnerability

In order to obtain the details of a vulnerability you can use the following request:

GET /v1/vulnerabilities/<vulnerability-id>

Path Parameters

- **vulnerability-id**: This is the internal ID of the vulnerability, as returned in the "id" field of the vulnerability list request's response.

Response

A successful request would return a JSON document with the following format:

```
{
   "id":"uuid",
   "organizationId":"string (UUID)",
   "status":"string",
   "data":[
      {
         "spid":"string",
         "name":"string",
         "description":"string",
         "remedy":"string",
         "cve":"string",
         "type":"string"
      }
   ],
   "assetId":"string (UUID)",
   "parents":[
      {
         "applicationId":"string (UUID)",
         "applicationName":"string",
         "componentName":"string",
         "componentId":"string (UUID)"
      }
   ],
   "scannerTargetId":"string (UUID)",
   "severityScore":"number",
   "exposed":"boolean",
   "location":"string",
   "dateFrom":"date",
   "dateTo":"date",
   "suggestedToFix":"boolean",
   "origin":"string",
   "epssScore":"decimal",
   "daysOpen":"number",
   "duplicate":"boolean",
   "scannerName":"string",
   "scannerType":"string",
   "hasFalsePositiveRequest":"boolean",
   "falsePositiveRequestDaysInQueue":"number",
   "assetLocality":"string",
   "slaPolicy":{
      "vulnerability":{
         "targetDays":"number",
         "targetDate":"date",
         "breach":"boolean"
```

```
      },
      "workflowTicket":{
         "targetDays":"number",
         "targetDate":"date",
         "breach":"boolean"
      }
   },
   "workflowTickets":[
      {
         "integrationWorkflowId":"string (UUID)",
         "workflowId":"string (UUID)",
         "workflowType":"string",
         "externalLink":"boolean",
         "externalName":"string",
         "externalUrl":"string",
         "details":"json",
         "createdDateTime":"datetime"
      }
   ],
   "tags":[
      {
         "key":"string",
         "value":"string",
         "category":"string"
      }
   ]

}
```

Import Endpoints

The import endpoints provide similar functionality to the Assessment Import area in the platform UI, but with a developer-friendly JSON format.

Import Assets

In order to import assets and vulnerabilities into Phoenix you can use the following endpoint:

POST /v1/import/assets

Parameters

```
{
  "importType": "new" | "merge" | “delta”,
  "assessment": {
    "assetType": "string",
    "name": "string"?,
  },
  "assets": [
    {
      "id": "string (UUID)",
      "attributes": // Attributes objects (see list of required attributes)
      "tags": Tag[],
      "installedSoftware": InstalledSoftware[]
      "findings": [
        // See list of findings fields
      ]
    }
  ]
}
```

where:

- **importType**: can be either “new”, “merge” or “delta”
- **assessment.assetType**: see table below for “Asset Type”'s
- **assessment.name**: The name for a new assessment (when “id” is null)
- **assets[].attributes**: List of asset details See table below for list of allowed fields for each type of asset.
- **assets[].findgins[]**: List of findings within each asset. See table below for list of allowed fields for each type of asset.

delta: The import contains information about vulnerabilities and assets that are present in the system at this moment. However, it might not include all the vulnerabilities in each asset, and might not include all the assets in the scope of the assessment. The platform will add or update assets and vulnerabilities, but **will not remove or alter** any assets or vulnerabilities not present in the report.

new, merge: The report includes all the vulnerabilities currently present in all the assets in the scope of the assessment. If an asset or vulnerability (that was present in a previous report for this assessment) is not present in the new report the platform assumes that those vulnerabilities have been addressed, and will close them. Assets and vulnerabilities present in the new report will be created or updated as required. “*new*” imports remove existing vulnerabilities from the assets before importing; “*merge*” update as required.
(**IMPORTANT**: Importing a delta report as a *new/merge* report **will close vulnerabilities** that might not have been intended to be closed by the user.)

Asset Attributes Fields

All fields are **strings** unless indicated otherwise.

Asset Type	Required fields	Optional fields
INFRA	**ip, hostname**	**network, fqdn, os, netbios, macAddress**
WEB	**ip, fqdn** (one of the two)	**ip, fqdn** (one of the two)
CLOUD	**providerType**: "AWS", "AZURE", "GCP" **providerAccountId**: e.g. AWS ARN **region**: only required for Azure' otherwise optional	**vpc, subnet, region** (for AWS, GCP) **providerAccountName**: user-friendly name **providerResourceId**: native resource ID from cloud provider **resourceGroup**: if available from the cloud provider
CONTAINER REPOSITORY CODE BUILD	**repository**: required for Repo **buildFile**: required for Build **dockerfile**: required for Container **scannerSource**: required for Code (JAR file, ZIP file, Snyk project,...)	**origin**: e.g. "github"

Findings Fields

All fields are strings unless indicated otherwise.

Required fields	Optional fields
name **description** **remedy** **severity**: "1.0" - "10.0" (string) **location**: required for CODE	**referenceIds**: array of CVE, GHSA and other IDs **cwes**: array of CWEs in the form "CWE-xxxx" **location**: e.g. source file or asset name **publishedDateTime**: "YYYY:MM:DD HH:MM:SS" (defaults to current date-time) **details**: JSON object with any additional details. They will be captured and displayed.

Tag

Required fields	Optional fields
value	**key**

Installed Software

Required fields	Optional fields
vendor **name** **version:** SemVer	**cpe**

Applications and Environments Endpoints

Applications/Environments List

In order to obtain the list of applications and environments you can use the following request:

GET /v1/applications

Query Parameters

Request page parameters:

```
?pageNumber=<0-based-page>&pageSize=<page-size>
```

Response

A successful request would return a JSON document with the following format:

```
{
   "content":[ Application objects ],
   "last":"boolean",
   "first":"boolean",
   "pageSize":"number",
   "pageNumber":"number",
   "pageNumberOfElements":"number",
   "totalPages":"number",
   "totalElements":"number"
}
```

Application Object

The structure of the application object in the response is as follows:

```
{
       "id":"uuid",
       "organizationId":"uuid",
       "name":"string",
       "owner":{
              "id":"uuid",
              "email":"string"
              },
       "criticality":"number",
       "value":"number",
       "risk":"number",
       "threshold":"number",
       "aboveThreshold":"boolean",
       "type":"string",
       "subType":"string",
       "responsibleUsers":[ {"id":"uuid", "email":"string"} ],
       "tags":[ {"key":"string", "value":"string"} ],
       "stats": {... see AppStats object below ...},
       "default":"boolean"
 }
```

In the returned application/environment data:

- **type**: Indicates if the entity is an Application or an Environment
- **subType**: For Environments only, it indicates if it's a Cloud or Infra environment. Otherwise it's null or missing.

AppStats Object

The structure of the AppStats object in the response is as follows:

```
{
  "findings": {
    "open": number,
    "closed": number,
    "openByRisk": {
      "critical": number,
      "high": number,
      "medium": number,
      "low": number,
      "none": number
    }
  },
  "assets": {
    "total": number
  }
}
```

Single Application/Environment

In order to obtain the details of an application or environment you can use the following request:

GET /v1/applications/<application-id>

A successful request to this endpoint would return a **single** Application object, as defined in the previous section.

Get Application/Environment Posture

In order to obtain the details of an application or environment you can use the following request:

GET /v1/applications/<application-id>/posture

Query Parameters

- **excludeRiskAccepted**: (boolean) whether the posture counts should exclude findings that have been "risk accepted". Defaults to false, so all findings are counted.

A successful request to this endpoint would return a **single** Application Posture object, as defined below.

```
{
       "id":"uuid",
       "organizationId":"uuid",
       "name":"string",
       "criticality":"number",
       "value":"number",
       "risk":"number",
        "riskMagnitude":"number",
       "threshold":"number",
       "aboveThreshold":"boolean",
       "type":"string",
        "default":"boolean"
       "posture": {
         "findings": {
           "open": number,
           "closed": number,
           "openByRisk": {
             "critical": number,
             "high": number,
             "medium": number,
             "low": number,
             "none": number
           }
         },
         "assets": {
           "total": number
         }
       },

 }
```

Alternatively, the Application can be identified through a "selector", using the following endpoint and parameters.

POST /v1/applications/posture

```
{
  "applicationSelector": {
       "name": "string",
       "caseSensitive": boolean (optional; defaults to true)
  },
  "excludeRiskAccepted": boolean
}
```

Create Application/Environment

To create a new application or environment you can use the following request:

POST /v1/applications

Parameters

Requests parameters are sent in the POST body:

```
{
   "name":"string",
   "type":"string",
   "subType": "string",
   "threshold":"number",
   "value":"number",
   "criticality":"number",
   "owner":{
       "id":"uuid",
       "email":"string"
   },
   "responsibleUsers":[
       {
       "id":"uuid",
       "email":"string"
       }
   ],
   "tags":[
       {
       "key":"string",
       "value":"string"
       }
   ]
}
```

where:

- **name**: Application name. Required field (3-255 characters long)
- **type**: Application Type. Required field. Allowed values: `APPLICATION, ENVIRONMENT`
- **subType**: Required for `ENVIRONMENT` application type. Allowed values: `CLOUD, INFRA`
- **threshold.** Optional field `(0-1000)`, otherwise inherited from Organization threshold
- **value**: Optional field `(1000-10000000)`
- **criticality:** Required field `(1-10)`
- **owner:** Required object. Either `id` or `email` must be provided
- **responsibleUsers[]:** Optional array of objects. Either `id` or `email` must be provided
- **tags[]**: Optional array of objects. A `key` is optional, `value` is required

Response

A successful request would return a JSON document with the created Application object.

Update Application/Environment

To update an existing application or environment you can use the following request:

PATCH /v1/applications

Parameters

Requests parameters are sent in the POST body:

```
{
   "name":"string",
   "threshold":"number",
   "value":"number",
   "criticality":"number",
   "owner":{
       "id":"uuid",
       "email":"string"
   }
}
```

Where:

- **name**: Optional. New application/environment name. Required field (3-255 characters long)
- **threshold.** Optional field `(0-1000)`, otherwise inherited from Organization threshold
- **value**: Optional field `(1000-10000000)`
- **criticality:** Optional field `(1-10)`
- **owner:** Optional object. If present, either `id` or `email` must be provided.

Response

A successful request would return a JSON document with the updated Application object.

Add Tags to Application/Environment

Tags can be added to applications and environments with the following request:

PUT /v1/applications/<application-id>/tags

Path Parameters

- **application-id**: This is the internal ID of the application, as returned in the "id" field of the application list request's response. Must not be the Default Application.

Parameters

Requests parameters are sent in the POST body:

```
{
   "tags":[
      {
         "key":"string",
         "value":"string"
      }
   ]
}
```

where:

- **tags[]**: Array of objects. A `key` is optional, `value` is required

Response

A successful request would return a JSON document with the updated Application object.

Remove Tags from Application/Environment

Tags can be **removed** from applications and environments by using the following endpoint.

PATCH /v1/applications/<application-id>/tags

Path Parameters

- **application-id**: This is the internal ID of the application, as returned in the "id" field of the application list request's response. Must not be the Default Application.. (See Identifying Applications by Name for an alternative.)

Parameters

Requests parameters are sent in the PATCH body:

```
{
   "action": "delete",
   "tags":[
      {
         "id": "uuid",
         "key": "string",
         "value": "string"
      }
   ]
}
```

where:

- **action:** Should be "***delete***".
- **tags[]:** Array of objects. `id` is optional, `key` is optional, `value` is required. If `id` is specified key/value would be ignored.

Response

A successful response will return the **204** HTTP Status Code without a response body. If no tags are matched for removal the request will return **404** Not Found.

Add responsible Users to an Application/Environment

With the following endpoint API clients can add responsible users to an application or environment. The users are **added to the existing list**, if not present already.

PUT /v1/applications/<application-id>/responsible-users

Path Parameters

- **application-id**: This is the internal ID of the application, as returned in the "id" field of the application list request's response. Must not be the **Default Application**.

Parameters

Requests parameters are sent in the POST body:

```
{
   "responsibleUsers":[
      {
         "id": "uuid",
         "email":"string"
      }
   ]
}
```

where:

- **responsibleUsers[]:** Array of user identification objects. Either `id` or `email` must be provided and they must already exist in the organisation.

Response

A successful request would return a JSON document with the updated Application object.

Add Deployment Link to an Application

With the following endpoint API clients can add deployment links to an application. If the application is already deployed, the new link will **replace** the previous one.

PATCH /v1/applications/<application-id>/deploy

Path Parameters

- **application-id**: This is the internal ID of the application, as returned in the "id" field of the application list request's response. It must be an **Application ID,** not an Environment's.

Parameters

Requests parameters are sent in the PATCH body:

```
{
   "serviceSelector":{
       "id": "uuid",
       "name": "string",
       "caseSensitive": boolean,
       "tags": [
         {
           "key": "string",
           "value": "string"
         }
       ],
       "matchAssetTags": boolean
   }
}
```

where **serviceSelector** contains:

- **id:** Optional**.** Phoenix identifier of the service where the application would be deployed. Either `id` or `name` must be provided and they must already exist in the organisation.
- **name**: Optional. The name of the service where the application would be deployed. If multiple services match the name, the first one would be used.
- **caseSensitive**: Optional. Whether "*name*" above should be searched in a case sensitive way. Defaults to **true**.
- **tags**: Optional. A list of tags that would be used to match the service to deploy the application onto. Selected services need to match at least ONE of the tags, and only the first matching service would be used.
- **matchAssetTags**: Optional. Whether to match services by the tags in their **assets**, as well as the service's own tags. The default value is ***true***.

Fields within serviceSelector offer different ways of identifying services within Phoenix. At least one needs to be present, and if more than one are provided **only** the first one in this order would be used: id, name, tags.

Response

A successful request would return a JSON document with the updated Application object.

If no service is found the response status is ***404*** *Not Found*.

Link Repository to an App/Env

This request can be used to add a "**repository rule**" to an application or environment. It will add a new rule to an existing component, or create a new component for the rule, depending on the parameters. This rule associates any asset with a "*repository*" attribute that matches the given name.

POST /v1/applications/<application-id>/repository

Path Parameters

- **application-id**: This is the internal ID of the application, as returned in the "id" field of the application list request's response. Must not be the **Default Application**.

Parameters

Requests parameters are sent in the POST body:

```
{
  "repository": "string",
  "search": "string",
  "component": {
    "id": "uuid",
    "name": "string",
    "criticality": number,
    "tags": [
      {
         "key": "string",
         "value": "string"
      }
    ]
  }
}
```

where:

- **repository:** Repository name. Required field. It's the name of the repository for the rule. (It will be used as part of the component name to create a new component if **componentName** is not provided.)
- **search**: Optional. It defines a search substring to match on the asset IDs while selecting assets from the matching *repository*. It can be used to select a subset of assets within the repository.
- **component**: Optional. Defines which component the repository (rule) should be added to within the application. If not present a new component would be created using the repository name.
    - **name:** Custom name to create a new component. Optional field. The name must be unique in the application. Required, unless "id" is present.
    - **id:** Provide specific existing component ID to link the rule to an existing component. Optional field. Must not be the **Default Component**. Required if "name" is not present.
    - **criticality**: Optional. A value between 1 and 10 to define the criticality of the component; it overrides the application's criticality.
    - **tags**: a list of key/value tags (key is optional) to add to the new or existing component.

If "***component***" is provided, the repository must not exist inside that component. Otherwise, a new component will be created and the repository must not exist inside the application.

Response

A successful request would return a JSON document with the updated Application object.

Identifying Applications/Environments by Name

The endpoints above that require a **<application-id>** parameter in the URL have an alternative version that uses the team's name instead.

To use the alternative version, use the same URL but removing the ***<application-id>*** segment, and add a team selector to the body of the request.

```
{
  ...

  "applicationSelector": {
      "name": "string",
      "caseSensitive": boolean (optional; defaults to true)
  }
}
```

In this case the platform will search the application/environment by name, optionally using a case insensitive matching.

Components and Services Endpoints

Component/Services List

In order to obtain the list of components and services you can use the following request:

GET /v1/components

Query Parameters

Request page parameters:

```
?pageNumber=<0-based-page>&pageSize=<page-size>
```

Response

A successful request would return a JSON document with the following format:

```
{
   "content":[
     {
       "id": "uuid",
       "organizationId": "uuid",
       "applicationId": "uuid",
       "name": "string",
       "criticality": number | null,
       "tags": [
         {
           "key":"string",
           "value":"string"
         }
       ],
       "default": false
     }
   ],
   "last":"boolean",
   "first":"boolean",
   "pageSize":"number",
   "pageNumber":"number",
   "pageNumberOfElements":"number",
   "totalPages":"number",
   "totalElements":"number"
}
```

Single Component/Service

In order to obtain the details of a component or service you can use the following request:

GET /v1/components/<component-id>

A successful request to this endpoint would return a **single** Component object, as defined in the previous section.

Get Component Posture

In order to obtain the details of a component or service you can use the following request:

GET /v1/components/<component-id>/posture

Query Parameters

- **excludeRiskAccepted**: (boolean) whether the posture counts should exclude findings that have been "risk accepted". Defaults to false, so all findings are counted.

A successful request to this endpoint would return a **single** Component Posture object, as defined below.

```
{
        "id":"uuid",
        "organizationId":"uuid",
         "applicationId":"uuid",
        "name":"string",
        "criticality":"number",
        "risk":"number",
        "riskMagnitude":"number",
         "default":"boolean"
        "posture": {
          "findings": {
            "open": number,
            "closed": number,
            "openByRisk": {
              "critical": number,
              "high": number,
              "medium": number,
              "low": number,
              "none": number
            }
          },
          "assets": {
            "total": number
          }
        },

 }
```

Alternatively, the Application can be identified through a “selector”, using the following endpoint and parameters.

POST /v1/components/posture

```
{
  "selector": {
        "applicationSelector": {
             "name": "string",
             "caseSensitive": boolean (optional; defaults to true)
        },
        "componentSelector": {
             "name": "string",
             "caseSensitive": boolean (optional; defaults to true)
        },
      },
  "excludeRiskAccepted": boolean
}
```

Create Components/Services

To create a new component or service you can use the following request:

POST /v1/components

Parameters

Requests parameters are sent in the POST body:

```
{
   "applicationId":"uuid",
   "name":"string",
   "criticality": number | null,
   "tags":[
       {
       "key":"string",
       "value":"string"
       }
   ]
}
```

where:

- **applicationId**: Required. Is the ID of the application where this component or service should be added.
- **name**: Required field (3-255 characters long)
- **criticality:** Optional field `(1-10)`
- **tags[]**: Optional array of objects. A `key` is optional, `value` is required.

Response

A successful request would return a JSON document with the created Component object.

Update a Component/Service

To update an existing component or service you can use the following request:

PATCH /v1/components/<component-id>

Path Parameters

- **component-id**: the ID (UUID) of the component to update

Parameters

Requests parameters are sent in the POST body:

```
{
   "name":"string",
   "criticality": number | null,
   "tags":[
       {
       "key":"string",
       "value":"string"
       }
   ]
}
```

where:

- **name**: Optional field (3-255 characters long)
- **criticality:** Optional field `(1-10)`
- **tags[]**: Optional array of objects. A `key` is optional, `value` is required. Tags will be added to the component, unless the same key/value combination is already present.

Response

A successful request would return a JSON document with the updated Component object.

Remove Tags from Component/Service

Tags can be **removed** from components and services by using the following endpoint.

PATCH /v1/components/<component-id>/tags

Path Parameters

- **component-id**: This is the internal ID of the component, as returned in the "id" field of the component list request's response. Must not be the Default Component or Service.. (See the "***selector***" parameter in Asset Association for an alternative.)

Parameters

Requests parameters are sent in the PATCH body:

```
{
   "action": "delete",
   "tags":[
      {
         "id": "uuid",
         "key": "string",
         "value": "string"
      }
   ]
}
```

where:

- **action:** Should be "***delete***".
- **tags[]:** Array of objects. `id` is optional, `key` is optional, `value` is required. If `id` is specified key/value would be ignored.

Response

A successful response will return the **204** HTTP Status Code without a response body. If no tags are matched for removal the request will return **404** Not Found.

Asset Association Rules API Endpoints

The following endpoints are used to manage the set of asset association rules that map assets to Components and Services.

Add Asset Association Rules

Add rule to Component/Service to associate assets to Components are Services. The endpoint is:

POST /v1/components/rules

Parameters

Requests parameters are sent in the POST body:

```
{
  "selector": {
      "applicationSelector": {
        "id": "string",
        "name": "string",
        "caseSensitive": true
      },
      "componentSelector": {
        "name": "string",
        "caseSensitive": true
      }
  },
  "rules": [
    {
      "name": "string",
      "filter": {
        "ids": ["uuid"],
        "keyLike": "string",
        "tags": [{
            "key": "string",
            "value": "string"
        }]
      // CLOUD =============
        "providerAccountId": ["string"],
        "providerAccountName": ["string"],
        "resourceGroup": [“string”],
      // INFRA =============
        "assetType": "CONTAINER" | "INFRA",
        "cidr": “cidr”,
        "hostnames": ["string"],
        "osNames": ["string"],
        "netbios": ["string"]
      // APPSEC =============
        "assetType": "REPOSITORY" | ”SOURCE_CODE" | "BUILD" | "WEBSITE_API",
        "cidr": "cidr",
        "fqdn": ["string"],
        "repository": ["string"]
      }
    }
  ]
}
```

where:

- **applicationSelector**: Required. Used to identify the Environment where the Service is located.
- **componentSelector**: Required. Used to identify the Service where the new rule(s) should be defined.
- **rules:** Required. List of rules defined as:
  - **name**: The rule's name
  - **filter**: A list of filters that define the rule - for now only "tags" filter is available.
    - **ids**: Array of asset IDs (UUIDs). If provided, these assets - and only these - are selected by the rule.
    - **keyLike**: Case sensitive search substring matched against the assets key. It should be surrounded by "*" characters: e.g. "*prod*".
    - **tags**: Array of objects. A `key` is optional, `value` is required.

For Cloud environments, the filter object can include:

- **providerAccountId**: The cloud provider's ID of the account or subscription where the assets belong.
- **providerAccountName**: The friendly name of the cloud provider's account or subscription.
- **resourceGroup**: The name of the resource group where the cloud asset belongs - if applicable.

For Infra environments, the filter object can include:

- **assetType**: Possible values are "INFRA" and "CONTAINER". If missing, all types would be included.
- **cidr**: A range of IP addresses defined as a valid CIDR value
- **hostnames**: List of hostnames to match
- **osNames**: List of operating system names to match
- **netbios**: List of NetBIOS names to match

For applications, the filter object can include:

- **assetType**: Possible values are "REPOSITORY", "SOURCE_CODE", "BUILD" and "WEBSITE_API". If missing, all types would be included.
- **cidr**: A range of IP addresses defined as a valid CIDR value
- **fqdn**: List of domain names to match.
- **repository**: List of repository names to match. Applies to all asset types except WEBSITE_API.

Important: This endpoint will prevent duplication of rules inside the same Component or Service. In order to do this, the rules are compared based on their definition (filters), not their name.

Response

A successful request would return an HTTP code **201 Created**, with the body of the response containing an indication of the number of rules added.

If all the rules are already present in the Component/Service, the platform would respond with HTTP code **409 Conflict**.

Team API Endpoints

List Teams

This request provides a list of user Teams in the organisation.

GET /v1/teams

Query Parameters

Request page parameters (optional, default values 0-100):

```
?pageNumber=0&pageSize=100
```

Response

A successful request would return a JSON document with the following format:

```
{
   "content":[
      {
         "id":"uuid",
         "name":"string",
         "type":"string",
         "usersCount": "number"
      }
   ],
   "last": "boolean",
   "first": "boolean",
   "pageSize": "number",
   "pageNumber": "number",
   "pageNumberOfElements": "number",
   "totalPages": "number",
   "totalElements": "number"

}
```

Single Team

This request provides details for a single Team.

GET /v1/teams/<team-id>

A successful request would return an object like this:

```
{
   "id":"uuid",
   "name":"string",
   "type":"string",
   "usersCount": "number"
}
```

Get Team Members

This request provides the list of members (users) in a Team.

GET /v1/teams/<team-id>/users

A successful request would return a list of objects like this:

```
[
  {
     "id":"uuid",
     "email":"string"
  }
]
```

Remove Team Member

This request removes a single member (user), identified by the **user-email** parameter, from a Team.

DELETE /v1/teams/<team-id>/users/<user-email>

Create Team

POST /v1/teams

Parameters

Requests parameters are sent in the POST body:

```
{
    "name": "string",
    "type": "string"
}
```

where:

- **name**: Required field. Name of the team. Must be unique in the organisation
- **type**: Required field. Team type. Allowed values: `GENERAL, SECURITY`

Response

A successful request would return a JSON document with the following format (Team object):

```
{
   "id":"uuid",
   "name":"string",
   "type":"string",
   "usersCount":"number"
}
```

Add users to a Team

This request adds new team members to a Team.

PUT /v1/teams/<team-id>/users

Path Parameters

- **team-id**: This is the internal ID of the team, as returned in the "id" field of the team list request's response. (See Identifying Teams by Name for an alternative.)

Parameters

Requests parameters are sent in the POST body:

```
{
   "users":[
      {
         "id": "uuid",
         "email":"string"
      }
   ],
   "autoCreateUsers": boolean
}
```

where

- **users[]:** Array of objects. Either `id` or `email` must be provided.
- **autoCreateUsers**: Optional boolean; defaults to "false". When set to "**true**", if users are identified by email and they are not present in the platform already they will be auto-created (with Org User role). Otherwise, the request will return code 400 "Invalid user email" for missing users.

Response

A successful response will return the **200** HTTP Status Code without a response body.

If *all* users in the *users[]* array are already members of the team the request will return a **409** HTTP status code. No changes will be made to the team.

If *any users[].id* or *users[].email* (without aoutCreateUsers enabled) value is not found in the platform, the request will return **400** HTTP status code.

Auto-link Apps/Envs to the Team by tags

With this endpoint clients can configure the auto-link tags used to link applications and environments to a team.

POST /v1/teams/<team-id>/applications/auto-link/tags

Path Parameters

- **team-id**: This is the internal ID of the team, as returned in the "id" field of the team list request's response. (See Identifying Teams by Name for an alternative.)

Parameters

Requests parameters are sent in the POST body:

```
{
   "match": "string",
   "tags":[
      {
         "id": "uuid",
         "key": "string",
         "value": "string"
      }
   ]
}
```

where:

- **match:** Is a tag match expression. Optional field. Allowed values: `ANY`, `ALL`. If not provided, and if the team doesn't have an auto-link rule, `ANY` is used. If not provided, and if the team has an existing auto-link rule, the existing rule's expression will be used.
- **tags[]:** Array of objects. `id` is optional, `key` is optional, `value` is required.

Response

A successful response will return the **200** HTTP Status Code without a response body.

If the *match* mode is not provided, or hasn't changed, and all *tags[]* are already in the team's configuration, the request will return **409** HTTP status code.

Remove applications auto-link tags

This request **removes** specific tags previously configured in the team for auto-linking to Apps/Envs. After invoking this request on a team the specified tag(s) will be removed from the team's auto-linking configuration.

PATCH /v1/teams/<team-id>/applications/auto-link/tags

Path Parameters

- **team-id**: This is the internal ID of the team, as returned in the "id" field of the team list request's response. (See Identifying Teams by Name for an alternative.)

Parameters

Requests parameters are sent in the PATCH body:

```
{
   "action": "delete",
   "tags":[
      {
         "id": "uuid",
         "key": "string",
         "value": "string"
      }
   ]
}
```

where:

- **action:** Should be "***delete***".
- **tags[]:** Array of objects. `id` is optional, `key` is optional, `value` is required. If `id` is specified key/value would be ignored.

Response

A successful response will return the **204** HTTP Status Code without a response body.

Remove auto-link applications to the team by tags

This request **removes** any tags previously configured in the team for auto-linking to Apps/Envs. After invoking this request on a team it will not have any auto-linking by tags configured.

DELETE /v1/teams/<team-id>/applications/auto-link/tags

Path Parameters

- **team-id**: This is the internal ID of the team, as returned in the "id" field of the team list request's response. (See Identifying Teams by Name for an alternative.)

Response

A successful response will return the **200** HTTP Status Code without a response body.

Enable auto-linking Apps/Envs to a Team by team Members

This request **enables** the auto-linking of the teams to the applications and environments that the team members are responsible for. This setting is shown as "*Auto-link based on Team Members*" in the UI.

POST /v1/teams/<team-id>/applications/auto-link/users

Path Parameters

- **team-id**: This is the internal ID of the team, as returned in the "id" field of the team list request's response. (See Identifying Teams by Name for an alternative.)

This will auto-link apps/envs to the team if team members are set as responsible users in the application.

Response

A successful response will return the **200** HTTP Status Code without a response body.

Disable auto-linking Apps/Envs to a Team by team Members

This request **disables** the auto-linking of the teams to the applications and environments that the team members are responsible for. This setting is shown as "*Auto-link based on Team Members*" in the UI.

DELETE /v1/teams/<team-id>/applications/auto-link/users

Path Parameters

- **team-id**: This is the internal ID of the team, as returned in the "id" field of the team list request's response. (See Identifying Teams by Name for an alternative.)

Response

A successful response will return the 200 HTTP Status Code without a response body.

Auto-link Components to Teams by Tags

With this endpoint clients can configure the auto-link tags used to link Components and Services to a team.

POST /v1/teams/<team-id>/components/auto-link/tags

Path Parameters

- **team-id**: This is the internal ID of the team, as returned in the "id" field of the team list request's response. (See Identifying Teams by Name for an alternative.)

Parameters

Requests parameters are sent in the POST body:

```
{
   "match": "string",
   "tags":[
      {
         "id": "uuid",
         "key": "string",
         "value": "string"
      }
   ]
}
```

where:

- **match:** Is a tag match expression. Optional field. Allowed values: `ANY`, `ALL`. If not provided, and if the team doesn't have an auto-link rule, `ANY` is used. If not provided, and if the team has an existing auto-link rule, the existing rule's expression will be used.
- **tags[]:** Array of objects. `id` is optional, `key` is optional, `value` is required.

Response

A successful response will return the **200** HTTP Status Code without a response body.

If the *match* mode is not provided, or hasn't changed, and all *tags[]* are already in the team's configuration, the request will return **409** HTTP status code.

Disable auto-linking Components to Teams by Tags

This request **removes** any tags previously configured in the team for auto-linking to Components (or Services). After invoking this request on a team it will not have any Component/Service auto-linking by tags configured.

DELETE /v1/teams/<team-id>/components/auto-link/tags

Path Parameters

- **team-id**: This is the internal ID of the team, as returned in the "id" field of the team list request's response. (See Identifying Teams by Name for an alternative.)

Response

A successful response will return the **200** HTTP Status Code without a response body.

Remove team's auto-link Tags

This request **removes** specific tags previously configured in the team for auto-linking to Apps/Envs or Components/Services. After invoking this request on a team the specified tag(s) will be removed from the team's auto-linking configuration, if they exist.

PATCH /v1/teams/<team-id>/applications/auto-link/tags

PATCH /v1/teams/<team-id>/components/auto-link/tags

Path Parameters

- **team-id**: This is the internal ID of the team, as returned in the "id" field of the team list request's response. (See Identifying Teams by Name for an alternative.)

Parameters

Requests parameters are sent in the PATCH body:

```
{
   "action": "delete",
   "tags":[
      {
         "id": "uuid",
         "key": "string",
         "value": "string"
      }
   ]
}
```

where:

- **action:** Should be "***delete***".
- **tags[]:** Array of objects. Requires `id` *or* `key/value`. (`id` takes precedence)

Response

A successful response will return the **204** HTTP Status Code without a response body. If none of the tags exist in the team configuration a **404** Not Found code would be returned.

Identifying Teams by Name

The endpoints above that require a **<team-id>** parameter in the URL have an alternative version that uses the team's name instead.

To use the alternative version, use the same URL but removing the ***<team-id>*** segment, and add a team selector to the body of the request.

```
{
  ...

  "teamSelector": {
      "name": "string",
      "caseSensitive": boolean (optional; defaults to true)
  }
}
```

In this case the platform will search the team by name, optionally using a case insensitive matching.

Users Endpoints

List Users

Provides a list of all users defined within the organisation.

GET /v1/users

Query Parameters

Request page parameters (optional, default values 0-100):

```
?pageNumber=0&pageSize=100
```

Response

A successful request would return a JSON document with the following format (User objects):

```
{
   "content":[
      {
         "id":"uuid",
         "email":"string",
         "firstName":"string",
         "lastName":"string",
         "active":"boolean",
         "role":"string"
      }
   ],
   "last":"boolean",
   "first":"boolean",
   "pageSize":"number",
   "pageNumber":"number",
   "pageNumberOfElements":"number",
   "totalPages":"number",
   "totalElements":"number"

}
```

Create User

Creates a new user within the organisation and sends the welcome and one-time-password emails to the provided address. This action is equivalent to creating a user through the Phoenix Security user interface.

POST /v1/user

Parameters

Requests parameters are sent in the POST body:

```
{
    "email": "string",
    "firstName": "string",
    "lastName": "string",
    "role": "string"

}
```

Where:

- **email:** User's email. Required field. Must be unique in the organisation.
- **firstName:** User's first name. Required field
- **lastName:** User's last name. Required field
- **role:** User's role. Required field. Allowed values: `ORG_ADMIN, ORG_APP_ADMIN, ORG_USER, ORG_ADMIN_LITE, ORG_SEC_ADMIN, ORG_SEC_DEV`

Response

A successful request would return an **200** HTTP status code and a JSON document with the following format (User object):

```
{
   "id":"uuid",
   "email":"string",
   "firstName":"string",
   "lastName":"string",
   "active":"boolean",
   "role":"string"
}
```

If a user already exists with the same ***email*** provided in the request, the response will be a **409** HTTP status code.

Support

E: premium.support@phoenix.security
P: +44(0)020 3195 3879
W: https://phoenix.security

Address

124 CITY ROAD
EC2V 2NX
LONDON, UK